Practical 5.1 - 0/1 Knapsack Problem

```
#include <iostream>
using namespace std;
int max(int a, int b) {
    return (a > b) ? a : b;
}
int main() {
    int n, W;
    cout << "Enter number of items: ";
    cin >> n;
    int wt[n], val[n];
    for(int i = 0; i < n; i++) {
        cout << "Enter weight and profit of item " << i + 1 << ": ";
        cin >> wt[i] >> val[i];
    }
    cout << "Enter knapsack capacity: ";
    cin >> W;
    int dp[n + 1][W + 1];
    for(int i = 0; i <= n; i++) {
        for(int w = 0; w <= W; w++) {
            if(i == 0 || w == 0)
                dp[i][w] = 0;
            else if(wt[i - 1] <= w)
                dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]], dp[i - 1][w]);
            else
                dp[i][w] = dp[i - 1][w];
        }
    }
    cout << "Maximum profit: " << dp[n][W];
```

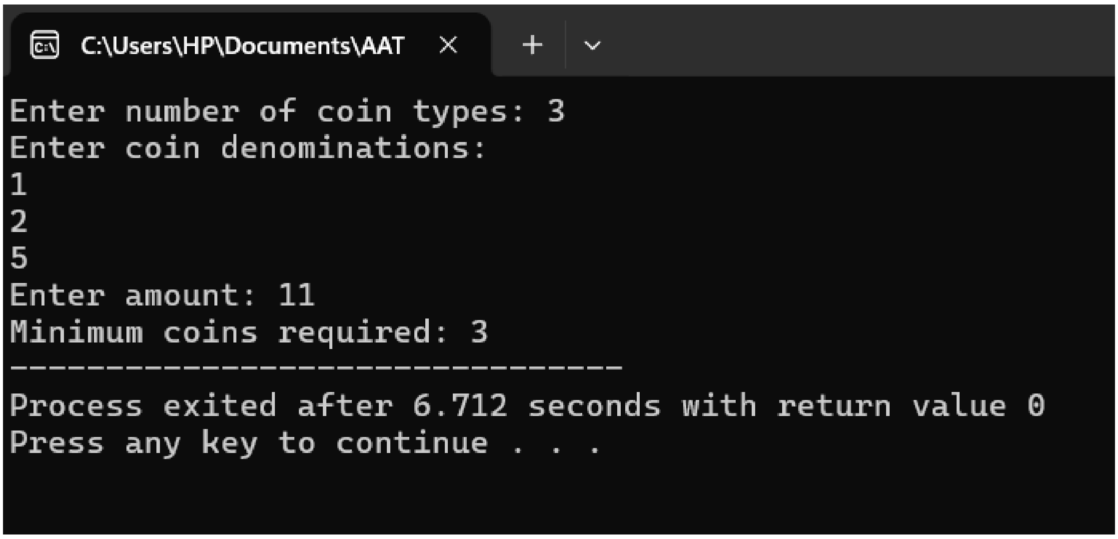
```
    return 0;
}
```

```
C:\Users\HP\Documents\AAT
Enter number of items: 3
Enter weight and profit of item 1: 10 60
Enter weight and profit of item 2: 20 100
Enter weight and profit of item 3: 30 300
Enter knapsack capacity: 50
Maximum profit: 400
--------------------------------
Process exited after 13.89 seconds with return value 0
Press any key to continue . . .
```

5.2 - Coin Change-Making Problem

```
#include <iostream>

#include <climits>

using namespace std;


int min(int a, int b) {

    return (a < b) ? a : b;

}


int main() {

    int n, amount;

    cout << "Enter number of coin types: ";

    cin >> n;


    int coins[n];

    cout << "Enter coin denominations:\n";

    for(int i = 0; i < n; i++)

        cin >> coins[i];


    cout << "Enter amount: ";

    cin >> amount;


    int dp[amount + 1];

    dp[0] = 0;


    for(int i = 1; i <= amount; i++)

        dp[i] = INT_MAX;


    for(int i = 1; i <= amount; i++) {
```

```
        for(int j = 0; j < n; j++) {
            if(coins[j] <= i && dp[i - coins[j]] != INT_MAX)
                dp[i] = min(dp[i], dp[i - coins[j]] + 1);
        }
    }

    cout << "Minimum coins required: " << dp[amount];

    return 0;
}
```



5.3 - Weighted Job Scheduling Problem

```
#include <iostream>
#include <algorithm>
using namespace std;

struct Job {
    int start, finish, profit;
};

bool compare(Job a, Job b) {
    return a.finish < b.finish;
}

int main() {
    int n;
    cout << "Enter number of jobs: ";
    cin >> n;

    Job jobs[n];

    for(int i = 0; i < n; i++) {
        cout << "Enter start, finish and profit: ";
        cin >> jobs[i].start >> jobs[i].finish >> jobs[i].profit;
    }

    sort(jobs, jobs + n, compare);

    int dp[n];
    dp[0] = jobs[0].profit;
```

```
    for(int i = 1; i < n; i++) {
        int incl = jobs[i].profit;

        for(int j = i - 1; j >= 0; j--) {
            if(jobs[j].finish <= jobs[i].start) {
                incl += dp[j];
                break;
            }
        }

        dp[i] = max(incl, dp[i - 1]);
    }

    cout << "Maximum profit: " << dp[n - 1];

    return 0;
}
```

```
C:\Users\HP\Documents\AAT
Enter number of jobs: 3
Enter start, finish and profit: 1 3 50
Enter start, finish and profit: 3 5 20
Enter start, finish and profit: 2 100 200
Maximum profit: 200
--------------------------------
Process exited after 50.68 seconds with return value 0
Press any key to continue . . .
```